University of Colombo School of Computing

SCS 1304 - Problem Solving Strategies and Computation Approaches

Lab Sheet 08

Brute Force, Divide-and-Conquer & Dynamic Programing

Answer all parts of this question. Show all calculations and intermediate steps where applicable. You may use a non-programmable calculator.

On the day of a national election in Sri Lanka, vote count records are transmitted from thousands of polling stations to the central election monitoring system. To provide timely and accurate election updates, the system must process, organize, and analyse incoming data efficiently. Different algorithmic techniques are used at different stages of the workflow depending on the nature of the task.

A. As vote count records arrive from 5,000 polling stations, they are stored according to their arrival time before being displayed on the public results dashboard. To perform this sorting efficiently, the development team decides to use Quick Sort.

Answer the following:

1. Explain the purpose of the pivot in the Quick Sort algorithm and describe how it partitions the dataset during each recursive step.
2. Using the dataset: **[245, 120, 375, 180, 320, 150],** demonstrate the first two recursive levels of the Quick Sort algorithm. Clearly identify the pivot selected at each stage and show how the list is partitioned.
3. Assuming the worst-case behaviour of Quick Sort, calculate the approximate number of comparisons required to sort 5,000 records. Show your calculation.
4. Suggest one practical technique that can reduce the likelihood of the worst-case performance in Quick Sort. Briefly justify your answer.

B. After the arrival times have been sorted, the records within each electoral district must be arranged according to Candidate ID before district-level vote aggregation begins. The team selects Merge Sort because of its predictable performance.

Answer the following:

1. Explain how the Divide and Conquer strategy is applied in Merge Sort by using the following dataset.

   [34, 27, 40, 21, 30, 25]

   Illustrate both the division and merging phases.

2. Estimate the approximate number of comparisons required to sort 1,000 records using the Merge Sort time complexity:

   $$T(n)=n\times\log_2(n)$$

   Show your calculations.

3. Suppose the election data is distributed across 10 servers, where each server processes a separate group of districts. Explain how parallel Merge Sort can improve the overall performance of the system.

C. To identify relationships between voting patterns in different regions, the election monitoring system performs matrix multiplication on vote correlation matrices. For a simplified example, consider the following two matrices:

Matrix A:

$$\begin{bmatrix} 3 & 2 \\ 1 & 4 \end{bmatrix}$$

Matrix B:

$$\begin{bmatrix} 5 & 1 \\ 2 & 3 \end{bmatrix}$$

Answer the following:

1. Apply Strassen's Matrix Multiplication Algorithm to compute the product of the two matrices. Show all seven intermediate products and the final matrix.

2. Compare the number of multiplication operations required by the standard matrix multiplication algorithm, and Strassen's algorithm for this example. Explain why Strassen's algorithm becomes more advantageous for larger matrices.

D. To provide live election updates, the Election Commission continuously estimates the number of possible ways vote summary reports can be transmitted from a polling station to the central server through a network of intermediate relay stations. During development, the software engineers observed that the recursive solution repeatedly solves the same subproblems. To improve efficiency, they decided to use **Dynamic Programming**.

Answer the following

1. Explain what is meant by **overlapping subproblems**. Why is Dynamic Programming more suitable than a simple recursive approach for problems that exhibit this property?
2. The number of possible report transmission paths is defined using the following recurrence relation: ***F(n)=F(n−1)+F(n−2)***; ***where F(1)=1, F(2)=2***. Using **Dynamic Programming (tabulation)**, calculate **F(8)**. Clearly show the table used to compute the solution.
3. Draw the recursion tree for **F(6)** using the recursive solution. Identify **at least three overlapping subproblems** that are computed more than once.
4. Compare the time complexity of the naive recursive solution and the Dynamic Programming solution for this problem. Briefly explain why Dynamic Programming is more efficient.